COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Module 07: Linear algebra and game geometry

Expanded Volume 02 / individual teaching module

Four lessons with derivations, runnable examples, practical work and 20 quiz questions.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Module 07 — Linear algebra and game geometry

Original prerequisites: Module 06.1; basic Python functions.

Original module project: Build a tested vector/matrix library for a two-dimensional scene, then demonstrate a three-dimensional camera transformation with written coordinate conventions.

Expanded project: A tested geometry and fitting toolkit. A small vector/matrix implementation with tests, a parent-child scene calculation, a camera coordinate conversion, one quaternion/matrix agreement case, and a five-point least-squares report.

Acceptance: Include zero-vector rejection, noncommuting transforms, a singular system, known hand-derived transforms, q/-q agreement, and residual/conditioning distinctions. Record column-vector order, handedness, axes, and homogeneous point/direction rules.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 07.1 — Vectors, dot products and projections

Learning outcome: Use vectors for direction, distance and component measurements.

Separate locations from displacements

A point locates something in a coordinate system. Subtracting two points produces a displacement vector: target minus origin. Adding a displacement to a point produces another point. A vector's components depend on the chosen basis; the geometric displacement need not change when those coordinates change.

In a two-dimensional x-right, y-up plane, moving from (2,1) to (5,5) produces displacement (3,4). Its Euclidean length is sqrt(3^2+4^2)=5. That length uses the ordinary orthonormal Cartesian metric. If one axis has been rescaled without accounting for units, blindly summing component squares may no longer measure the intended physical distance.

A direction is a vector divided by its length

For nonzero v, its unit direction is u=v/||v||. Multiplying u by a speed creates a velocity with that speed. This avoids diagonal movement becoming faster merely because both input axes are active: (1,1) has length sqrt(2), not one.

The zero vector has length zero and no unique direction. Decide what it means in the application: stop moving, retain a prior orientation, or reject the direction request. The reference normalization function rejects it. A caller that wants a stationary fallback must handle that state explicitly rather than replacing it with an arbitrary direction deep inside the math library.

A direct sum of squares can overflow for very large components or underflow for very small ones. The reference first divides by the largest absolute component, normalizes that scaled vector, and therefore avoids squaring the original extremes. This is numerical design, not a change to the vector's intended direction.

The dot product measures a component

The dot product a·b is the sum of matching component products. It is a scalar. For nonzero vectors, a·b = ||a|| ||b|| cos(theta). A positive dot product means an acute angle; a negative one means an obtuse angle; zero means perpendicular directions when neither vector is zero.

For a unit direction u, the signed scalar component of v along u is v·u, and the vector projection is (v·u)u. For a nonunit direction b, the projection is ((v·b)/(b·b))b, provided b is nonzero. The omitted denominator is a common bug when an interface does not actually guarantee normalized input.

A front-facing test uses displacement d=target-position and facing f. The sign of f·d works without normalizing nonzero f. An angular field-of-view threshold needs lengths: f·d >= ||f|| ||d|| cos(theta_max). A target exactly at the observer has zero displacement and needs an explicit policy rather than a fabricated angle.

Orientation in three dimensions

In right-handed three-dimensional coordinates, the cross product a×b is perpendicular to both vectors, with components (ay*bz-az*by, az*bx-ax*bz, ax*by-ay*bx). Its length is the parallelogram area ||a|| ||b|| sin(theta). Swapping the operands negates the result. Parallel vectors produce zero, so they cannot determine a unique normal by normalization.

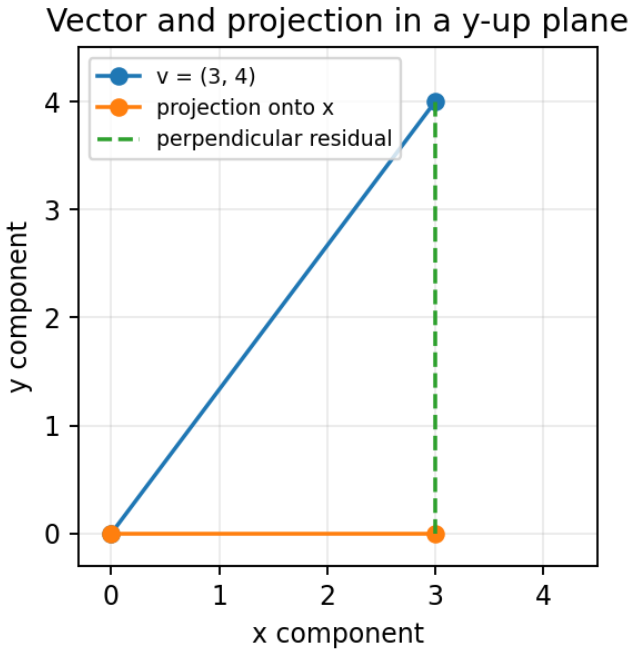
These operations answer different questions: dot gives alignment, cross gives an oriented perpendicular, and subtraction gives displacement. A good function name and type contract should make that distinction visible before the renderer or physics solver depends on it. [R07, R23]

Worked derivation

For v=(3,4), ||v||=5 and normalize(v)=(0.6,0.8). Its squared length is 0.36+0.64=1. Project v onto the x axis u=(1,0): v·u=3, so the projection vector is (3,0).

Project the same v onto b=(2,0), which is not unit length. The correct factor is (v·b)/(b·b)=6/4=1.5, giving 1.5*(2,0)=(3,0). Multiplying (v·b)b without the denominator would produce (12,0), four times the intended projection.

For facing (1,0), targets at displacements (2,1), (-2,1), and (0,4) are respectively in front, behind, and exactly to the side by the sign test.



Projection of (3,4) onto the x axis; the perpendicular residual is the y component.

Runnable demonstration

Run `python run_example.py 07.1` from the extracted package folder. Predict before executing.

```
from mathkit import dot, normalize
v = (3, 4)
u = normalize(v)
print("unit:", tuple(round(x, 3) for x in u))
print("x component:", dot(v, (1, 0)))
print("squared length:", dot(v, v))
try:
    normalize((0, 0))
except ValueError as error:
    print("rejected:", error)
```

Recorded expected output:

```
unit: (0.6, 0.8)
x component: 3.0
squared length: 25.0
rejected: zero vector has no unique direction
```

Practical assignment

Original brief: Implement length, dot and safe normalization. Use dot products to determine whether a target is in front of a character. Pass: test zero vectors, perpendicular vectors and opposite directions.

1. Implement dot(a,b) and normalize(v), documenting finite inputs, equal dimensions, and a deliberate zero-vector policy. Do not mutate the arguments.
2. Test perpendicular, opposite, and identical directions; include zero and extreme finite magnitudes in normalization.
3. Use target-position, not the target point alone, in a front-facing query. Decide what happens when target equals position.
4. Prove that the projection residual is perpendicular to a nonzero projection direction, then check a numerical case.

Coding slice: Work in `labs/student/lab_07_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Zero dot product does not establish two useful perpendicular directions when one operand is zero. Do not confuse a scalar component with a projection vector.

Independent transfer task

Find the projection of v=(2,2) onto b=(1,1), and then the rejection v-projection.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the length of (6,8)?
2. What is the dot product of (1,0) and (0,1)?

3. Why cannot the zero vector be normalized by the ordinary formula?
4. Why does unnormalized input (1,1) make diagonal movement faster than input (1,0) at the same scalar multiplier?
5. Compute the projection of (3,4) onto the nonunit vector (2,0).

Assigned study

R07: vectors, orthogonality, and projections. R23: vectors and coordinate systems. Complete both the analytic projection derivation and executable tests.

Original reading assignment retained: R07: vectors and projections; R23: Geometry and Transformations / Vectors.

Lesson 07.2 — Matrices, bases and linear systems

Learning outcome: Understand a matrix as a transformation rather than a table to memorize.

Read a matrix through its basis images

Using column vectors, a matrix's columns describe where the basis directions go. The matrix [[2,0],[0,1]] maps the x basis to (2,0) and the y basis to (0,1). Therefore (x,y) maps to (2x,y). This interpretation makes the multiplication rule a consequence of linearity rather than an arbitrary table procedure.

A linear transformation preserves addition and scalar multiplication. Translation alone is not linear in ordinary coordinates because it moves the zero vector; the next lesson adds homogeneous coordinates to represent translation in an augmented space. A basis contains independent vectors spanning the space. In two dimensions, two parallel nonzero vectors are not a basis: they cannot express displacements off their common line.

Composition has an order

For column vector p, applying A and then B gives B(Ap)=(BA)p. Read the rightmost factor first. The product entry (BA)[i,j] is a sum over matching inner indices. A matrix with r rows and k columns can multiply one with k rows and c columns; the result has r rows and c columns.

A rotation followed by a nonuniform scale usually differs from the reversed order. This is not a floating-point error: the exact transformations differ. Associativity, (AB)C=A(BC), permits regrouping in exact arithmetic, but does not permit swapping factors. Floating-point regrouping can also change the last few bits.

Solve equations without treating inverse as the default operation

For Ax=b, Gaussian elimination adds multiples of one equation to another to remove a variable. It transforms the system while preserving its solution set. A pivot is the coefficient used for elimination. Swapping rows to choose a larger available pivot reduces some avoidable numerical trouble, but cannot cure a fundamentally sensitive problem.

For 2x+y=5 and x-y=1, subtract half of the first equation from the second: -1.5y=-1.5. Thus y=1 and x=2. Verify both original equations, rather than trusting the intermediate arithmetic alone.

The reference solve2 routine scales the matrix, uses a row swap when useful, and rejects pivots that are small relative to its chosen scale. This is a stated numerical policy, not a proof that every rejected matrix is exactly singular. A different tolerance changes the supported problem envelope.

Rank and lost information

A singular square matrix lacks a unique inverse. A zero scale on one axis collapses distinct points together. More generally, a nonzero vector n with An=0 lies in the nullspace: x and x+n map to the same output. Rank counts independent output directions. A two-by-two matrix of rank one maps the plane into a line.

A singular system may have no solution or infinitely many solutions, depending on b. For equations x+y=2 and 2x+2y=4, every point on one line works. Replacing the second right side with 5 makes the equations inconsistent. "The solver failed" is therefore not a complete mathematical diagnosis.

If B contains basis vectors as columns, B*c converts coordinates c in that basis into the original basis. Recovering c means solving B*c=p. Document what each coordinate tuple means before combining transforms from different parts of an engine. [R07]

Worked derivation

Let R=[[0,-1],[1,0]] be a 90-degree counterclockwise rotation and S=[[2,0],[0,1]] a nonuniform scale. Start with p=(1,1).

Apply R first: Rp=(-1,1); then S gives SRp=(-2,1). Apply S first: Sp=(2,1); then R gives RSp=(-1,2). These exact outputs differ, demonstrating SR != RS.

For A=[[2,1],[1,-1]] and b=(5,1), elimination gives x=(2,1). The residual Ax-b is (0,0) in exact arithmetic. In numerical work, report a norm of that residual and the problem scale; a small residual alone will not later guarantee small solution error in an ill-conditioned problem.

Runnable demonstration

Run `python run_example.py 07.2` from the extracted package folder. Predict before executing.

```
from mathkit import matmul, solve2
rotation = ((0, -1), (1, 0))
scale = ((2, 0), (0, 1))
p = ((1,), (1,))
print("rotate then scale:", matmul(matmul(scale, rotation), p))
print("scale then rotate:", matmul(matmul(rotation, scale), p))
print("solve:", solve2(((2, 1), (1, -1)), (5, 1)))
```

Recorded expected output:

```
rotate then scale: ((-2.0,), (1.0,))
scale then rotate: ((-1.0,), (2.0,))
solve: (2.0, 1.0)
```

Practical assignment

Original brief: Implement 2-by-2 multiplication and solve two small linear systems by elimination on paper. Pass: show a pair of transforms that do not commute and explain a singular example geometrically.

1. Implement matmul(a,b) and solve2(a,b), with dimension checks and documented singular/small-pivot rejection. Test a row-swap case.
2. Show the rotation/scale noncommutativity example entirely on paper before checking it in code.
3. Solve two small linear systems by elimination and distinguish a unique, inconsistent, and underdetermined case.
4. Provide a nonzero nullspace vector for a singular matrix and verify that it explains two inputs with the same output.

Coding slice: Work in `labs/student/lab_07_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Row storage order in memory does not determine whether the mathematics uses row or column vectors. Keep representation, multiplication convention, and composition order separate.

Independent transfer task

Solve x+y=2 and 2x+2y=5. What fails?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Do matrices generally satisfy AB = BA?
2. What happens to invertibility when an axis is scaled to zero?
3. Why solve Ax=b without explicitly forming an inverse?
4. What are the output dimensions of a 3x2 matrix times a 2x4 matrix?
5. Give two different points that a zero x-scale maps to the same point.

Assigned study

R07: matrix multiplication, elimination, inverses, column space, nullspace, and change of basis. Work an elimination problem without forming an inverse.

Original reading assignment retained: R07: matrix multiplication, elimination, inverses and vector spaces.

Lesson 07.3 — Homogeneous transforms, cameras and quaternions

Learning outcome: Compose a scene using explicit coordinate conventions.

Write the coordinate contract first

This bundle uses column vectors, right-handed three-dimensional coordinates, x-right, y-up, and an active counterclockwise rotation about +z when viewed toward the origin from +z. A transform multiplies on the left. These are chosen conventions, not universal engine rules. The earlier terminal game's down-positive grid is a different coordinate system and needs an explicit conversion.

In homogeneous coordinates, represent a point as (x,y,z,1) and a direction as (dx,dy,dz,0). The affine transform with linear part A and translation t maps (p,1) to (Ap+t,1), and (d,0) to (Ad,0). The last row is (0,0,0,1). A translation must not change a pure direction.

Parent-child transforms are coordinate maps

Let M_parent convert parent-local coordinates into world coordinates, and M_child convert child-local coordinates into parent coordinates. Then M_world=M_parent*M_child. Each matrix has an input space and an output space. A chain is meaningful only when adjacent spaces match.

For a rigid pose, M=T(c)R means rotate local geometry by R, then translate by c. The inverse is R^T*T(-c), because an orthonormal rotation's inverse is its transpose. Reversing each operation without reversing their order gives the wrong inverse. Nonuniform scaling is not inverted by simply transposing the whole linear block.

The camera is another coordinate frame

A camera pose C maps camera-local coordinates into world space. The view matrix V=C^-1 maps world geometry into camera coordinates. A camera moving right makes fixed world geometry appear left in camera space; this is a change of viewpoint, not a modification to all world objects.

A perspective projection is a separate transformation. Its homogeneous output generally requires division by w, and clipping rules determine which points are representable or visible. This lesson's reference transform4 deliberately performs only the four-component multiply. It does not silently divide, clip, or impose a graphics API's depth range. Full perspective and clipping arrive in Volume 5.

Quaternions encode rotations, not positions

A unit quaternion in scalar-first form q=(w,x,y,z) represents an orientation. For a unit axis u and angle theta, q=(cos(theta/2), u*sin(theta/2)). The rotated vector can be obtained from q*(0,v)*q^-1, using quaternion multiplication. A quaternion has four components but represents a three-dimensional rotation with a unit-length constraint.

Both q and -q represent the same rotation. Consequently, interpolating components without handling that sign can take an unintended path or even produce the zero quaternion. A common normalized-linear interpolation first chooses the sign giving a nonnegative dot product, blends, and normalizes; it generally does not yield constant angular speed. Spherical interpolation addresses a different speed property and still needs consistent conventions.

Verification must challenge the convention

Test identity, a known quarter turn, translation of a point, translation of a direction, and a parent rotation that moves a child's offset. For an inverse round trip, compare within an absolute and relative tolerance tied to the numerical scale. A round trip by itself is weak: two implementations can share the same convention mistake and cancel it. Keep hand-derived expected positions as independent checks. [R07, R23]

Worked derivation

Take parent pose T(10,0,0)*Rz(pi/2) and child-local translation T(2,0,0). The child's origin first becomes (2,0,0) in parent coordinates, rotates to (0,2,0), and translates to (10,2,0) in world coordinates.

For T(10,20,30), point (1,0,0,1) becomes (11,20,30,1), but direction (1,0,0,0) stays unchanged. The difference follows directly from multiplication by the last component w.

For a z-axis quarter turn, q=(sqrt(2)/2,0,0,sqrt(2)/2). It rotates (1,0,0) to (0,1,0), agreeing with the matrix calculation. Test -q as well; the result should agree.

Runnable demonstration

Run `python run_example.py 07.3` from the extracted package folder. Predict before executing.

```
import math
from mathkit import translation, rotation_z, matmul, transform4, quat_rotate
parent = matmul(translation(10, 0, 0), rotation_z(math.pi / 2))
child = translation(2, 0, 0)
world = matmul(parent, child)
print("child origin:", tuple(round(x, 6) for x in transform4(world, (0, 0, 0, 1))))
print("direction:", transform4(translation(10, 20, 30), (1, 0, 0, 0)))
q = (math.cos(math.pi / 4), 0, 0, math.sin(math.pi / 4))
print("quaternion:", tuple(round(x, 6) for x in quat_rotate(q, (1, 0, 0))))
```

Recorded expected output:

```
child origin: (10.0, 2.0, 0.0, 1.0)
direction: (1.0, 0.0, 0.0, 0.0)
quaternion: (0.0, 1.0, 0.0)
```

Practical assignment

Original brief: Create a parented object and camera using documented conventions. Pass: test identity, inverse round trips within tolerance, a translated direction that remains unchanged, and a parent rotation that moves its child correctly.

1. Implement transform4(m,v) for a 4x4 matrix and a four-component column vector. It must not perform perspective division.
2. Create the parent-child chain above, a camera inverse round trip, and an independently derived quarter-turn test.
3. Verify points and directions have different translation behavior. Write an explicit grid-to-world conversion for y-down versus y-up.

4. Derive one axis-angle quaternion and compare its rotation with a matrix; check q and -q and reject the zero quaternion.

Coding slice: Work in `labs/student/lab_07_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not mix the terminal grid y-down convention with the geometry y-up convention, treat a direction as w=1, or confuse the camera pose with its inverse view transform.

Independent transfer task

A camera has no rotation and sits at (0,0,5). Where does world point (1,2,3) appear in camera coordinates?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why commonly use w=0 for directions?
2. What does the view transform do?
3. Why document multiplication order even when using a library?
4. Why do q and -q require care during quaternion interpolation?
5. For a rigid pose C=T(c)R, what is the view transform?

Assigned study

R23: coordinate systems and transformations, points, vectors, and rotations. R07: orthonormal matrices and inverses. The quaternion material is a new Package 03 expansion; its conventions are explicitly specified here.

Original reading assignment retained: R23: Coordinate Systems and Transformations; R19: geometric transformations.

Lesson 07.4 — Eigenvalues, least squares and singular values

Learning outcome: Recognize stability and approximation problems in game data.

An eigenvector identifies a preserved direction

For a square matrix A, a nonzero vector v is an eigenvector when Av=lambda*v. Lambda is its eigenvalue. The transformed vector remains on the same line, though a negative eigenvalue reverses orientation. Zero is not an eigenvector because it would make every lambda trivially satisfy the equation.

For diag(2,0.5), the coordinate axes are eigenvector directions. Repeated application multiplies the x component by 2^n and the y component by 0.5^n. One grows while the other decays. Not every real matrix has real eigenvectors: a planar 90-degree rotation has no preserved real line. Complex eigenvalues later help describe oscillatory systems.

Eigenvalues help analyze repeated updates, but a statement about all long-term behavior needs assumptions about the matrix and its eigenstructure. In particular, unit-magnitude eigenvalues alone do not always guarantee bounded powers when a matrix has nontrivial Jordan structure. This bundle uses explicitly derived small examples rather than a blanket stability slogan.

Fit a line by defining the error you minimize

For observed pairs (xi,yi), ordinary least squares with intercept chooses slope a and intercept b to minimize S(a,b)=sum(yi-a*xi-b)^2. It assumes this squared vertical residual is the intended objective. It is not minimizing distance perpendicular to the line, absolute errors, or the largest error.

Set the partial derivatives to zero. The intercept equation makes the average residual zero, giving b=mean(y)-a*mean(x). Substitute into the slope equation to obtain a=Sxy/Sxx, where Sxy=sum((xi-mean(x))*(yi-mean(y))) and Sxx=sum((xi-mean(x))^2). A unique slope needs Sxx>0: the predictor must vary.

A good fit does not prove a physical law. Residuals should be examined for structure, outliers, and changing spread. If all measurements use one time value, additional precision in arithmetic cannot invent missing time variation.

Singular values measure stretching

For a real matrix, singular value decomposition writes A=U*Sigma*V^T, with orthonormal columns in the appropriate factors and nonnegative singular values. Geometrically, V^T rotates or reflects input coordinates, Sigma stretches along principal directions, and U maps those directions into the output space. Singular values are the square roots of eigenvalues of A^T A.

For diag(3,-2), eigenvalues are 3 and -2, while singular values are 3 and 2. Singular values describe stretch magnitudes, not signed orientation. A zero singular value signals a lost direction. For an invertible square matrix, the two-norm condition number is sigma_max/sigma_min.

Separate conditioning, algorithms, and coding mistakes

Conditioning asks how much the exact solution can change when the problem data changes. Algorithmic stability asks how much extra error the numerical method introduces. A coding error is an implementation defect, such as using the wrong column. They require different repairs.

Consider A=diag(1,0.000001). Its condition number is one million. A tiny change to the second component of b can produce a much larger absolute change in the second component of x. A small residual Ax-b therefore does not automatically imply a small forward error in x. Centering data and using an appropriate solver can reduce avoidable numerical error, but cannot eliminate missing information or the problem's intrinsic sensitivity. [R07]

Worked derivation

For x=(0,1,2,3,4) and y=(1.1,2.9,5.2,6.8,9.0), mean(x)=2 and mean(y)=5. Sxx=10 and Sxy=19.7, so slope=1.97 and intercept=1.06.

The residuals y-(1.97x+1.06) are (0.04,-0.13,0.20,-0.17,0.06); they sum to zero apart from floating-point rounding. The sum of squared residuals is 0.091. That is evidence about this chosen squared-error fit, not proof that future movement follows a straight line.

Apply diag(2,0.5) four times to (1,1): the result is (16,0.0625). The eigenvalues explain the growth and decay without executing all four updates.

Runnable demonstration

Run `python run_example.py 07.4` from the extracted package folder. Predict before executing.

```
from mathkit import fit_line
xs = (0, 1, 2, 3, 4)
ys = (1.1, 2.9, 5.2, 6.8, 9.0)
slope, intercept = fit_line(xs, ys)
print(f"line: y = {slope:.3f} x + {intercept:.3f}")
print("residuals:", [round(y - (slope*x + intercept), 3) for x, y in zip(xs, ys)])
x, y = 1.0, 1.0
for _ in range(4):
    x, y = 2*x, 0.5*y
print("four diagonal updates:", (x, y))
```

Recorded expected output:

```
line: y = 1.970 x + 1.060
residuals: [0.04, -0.13, 0.2, -0.17, 0.06]
four diagonal updates: (16.0, 0.0625)
```

Practical assignment

Original brief: Fit a line to five noisy position/time observations and report residuals. Analyze repeated multiplication by a diagonal matrix. Pass: distinguish data noise, poor conditioning and a programming error in your report.

1. Implement fit_line(xs,ys), returning slope then intercept, and reject unequal lengths, fewer than two pairs, nonfinite data, and a constant predictor.
2. Fit the five supplied measurements and report every residual, their sum, and the squared-error objective.

3. Analyze powers of two diagonal matrices symbolically and compare with code; include a negative eigenvalue.
4. Construct a diagonal sensitivity example and distinguish noise, conditioning, and a deliberately introduced indexing defect.

Coding slice: Work in `labs/student/lab_07_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Singular values are not generally signed eigenvalues. A tiny residual, visually good fit, or stable implementation cannot by itself remove an ill-conditioned problem.

Independent transfer task

For diag(4,0.25), find the singular values, two-norm condition number, and two updates of (1,1).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What are the eigenvalues of diag(3,2)?
2. What objective does ordinary least squares minimize?
3. Can a stable algorithm make an ill-conditioned problem insensitive?
4. Can a 90-degree planar rotation have a nonzero real eigenvector?
5. Why does fitting a slope fail when every xi is equal?

Assigned study

R07: eigenvalues, diagonalization, least squares, and singular value decomposition. Derive the centered line-fit equations before using the reference routine.

Original reading assignment retained: R07: eigenvalues, least squares and singular value decomposition.